南京大学《计算机图形学》课程大作业
2026 年春季学期
2026 年 6 月 30 日
吴与同，231220066
yutong-wu@smail.nju.edu.cn

系统报告

基于 Python 和 PyQt5 的二维图形绘制系统

系统设计、算法实现与测试报告

摘要

本项目实现了一个完整的二维图形绘制系统，包含核心算法模块、命令行界面和图形交互界面。系统支持线段、多边形、椭圆、Bezier 曲线、三次均匀 B-spline 曲线的生成，支持平移、旋转、缩放以及 Cohen-Sutherland 和 Liang-Barsky 线段裁剪、Sutherland-Hodgman 多边形裁剪。命令行程序能够读取课程规定格式的指令文件并保存 BMP 位图；GUI 程序基于 PyQt5 实现，提供现代化浅色界面、图标命令工具栏、左侧分段工具箱、中央画布工作区、右侧图层/属性/命令面板、鼠标绘制、对象选择、拖拽编辑、控制点编辑、右键上下文菜单、交互式旋转/缩放、交互式线段及多边形裁剪、撤销重做、图层管理与拖拽排序、命令导入导出及编辑器、算法过程演示（含 DDA/Bresenham/椭圆/Bezier/Cohen-Sutherland/Liang-Barsky/扫描线填充/Wu 反走样 8 种）、算法并排对比（含性能计时）、多边形扫描线填充、图层遮挡管理、详细操作字幕实时反馈、画布缩放平移与自适应网格、网格吸附、画布概览小地图和操作历史记录导出等功能。

目录

1 系统目标与完成情况 **3**

2 系统架构设计 **4**

2.1 总体结构 4
2.2 设计取舍 6
2.3 数据流 6
2.4 坐标系约定 6
2.5 场景状态与接口设计 7

3 核心算法实现 **7**

3.1 线段生成算法 7
3.1.1 DDA 算法 7
3.1.2 Bresenham 算法 8

3.2 多边形绘制 8
3.3 中点椭圆生成算法 8
3.4 曲线生成算法 8
3.4.1 Bezier 曲线 8
3.4.2 三次均匀 B-spline 曲线 9
3.5 几何变换 9
3.6 线段裁剪 9
3.6.1 Cohen-Sutherland 算法 9
3.6.2 Liang-Barsky 算法 9
3.7 算法对比与适用场景 10

4 命令行界面实现 10
4.1 指令复现示例 11

5 图形交互界面实现 11
5.1 界面布局 11
5.2 绘制交互 12
5.3 对象选择、属性面板与编辑 12
5.4 交互式变换与视图反馈 13
5.5 模板、演示与报告导出 13
5.6 命令编辑器与操作历史 14

6 测试与结果 14
6.1 测试方法 14
6.2 测试结果图 14
6.3 测试覆盖分析 16

7 遇到的问题与解决方案 17
7.1 坐标系一致性 17
7.2 曲线采样断裂 18
7.3 CLI 与 GUI 逻辑重复 18
7.4 撤销粒度 18

8 扩展功能与系统亮点 18

9 最终交付与复现方式 19

10 个人思考 19

11 结论 20

1 系统目标与完成情况

本课程大作业要求使用 Python3 实现一个绘图系统，包括核心算法模块、命令行界面和图形交互界面。系统应支持课程规定的图元生成、几何变换和线段裁剪，并能通过文本指令文件复现绘制过程。

本项目围绕以下目标完成：

- 在 `cg_algorithms.py` 中集中实现核心图形学算法，且该文件只依赖 Python 标准库 `math`。
- 在 `cg_cli.py` 中实现命令行渲染流程，支持读取指令、维护场景状态、调用算法并保存 BMP 图像。
- 在 `cg_gui.py` 及相关模块中实现交互式绘图系统，覆盖课程要求的全部指令功能。
- 在 GUI 中进一步扩展可用性和展示性，包括现代化布局、分组图标按钮、左侧分段工具箱（含算法选择弹出菜单）、画布工作区、右键菜单、属性坐标表格、带行号命令编辑器、交互式变换（支持中心点拖拽）、撤销重做、图层管理与拖拽排序、模板场景、算法过程演示、算法对比、裁剪算法对比、多边形交互裁剪、扫描线填充、图层遮挡管理、画布概览小地图、详细操作字幕实时反馈和操作历史导出。
- 提供回归测试脚本，验证示例输入能够稳定生成非空图像，并生成可视化测试报告。

表 1 总结了系统对课程要求的覆盖情况。

表 1: 课程要求功能覆盖情况

功能	CLI	GUI	说明
重置画布、设置颜色、保存画布	支持	支持	CLI 保存 BMP；GUI 额外支持 PNG/JPG。
线段绘制	支持	支持	支持 DDA 和 Bresenham；GUI 中额外保留 Naive 作为演示扩展。
多边形绘制	支持	支持	逐边调用线段算法，支持 DDA 和 Bresenham。
椭圆绘制	支持	支持	使用中点椭圆生成算法，支持对角点任意顺序输入。
曲线绘制	支持	支持	支持 Bezier 和三次均匀 B-spline。
平移、旋转、缩放	支持	支持	椭圆不执行旋转，符合题目要求。
线段裁剪	支持	支持	支持 Cohen-Sutherland 和 Liang-Barsky。
多边形裁剪	支持 (`clipPolygon`)	支持	Sutherland-Hodgman 多边形矩形裁剪，CLI 和 GUI 均支持。
指令导入导出	支持读取	支持双向	GUI 可加载指令文件，也可导出可复现指令。
填充与反走样（扩展）	无	支持	扫描线填充与 Wu 反走样线段，作为 GUI 扩展功能演示。

2 系统架构设计

2.1 总体结构

系统采用分层结构，尽量将算法、场景状态和界面交互分离。主要模块如下：

表 2: 系统主要模块

模块	作用
`cg_algorithms.py`（入口）	核心算法模块入口，内部实现拆分在 `_cg_algorithms/` 子包中，包含线段生成（Naive/DDA/Bresenham/Wu）、多边形生成与填充、椭圆生成、曲线生成（Bezier/B-spline）、几何变换、线段裁剪（Cohen-Sutherland/Liang-Barsky）、多边形裁剪（Sutherland-Hodgman）和算法过程追踪等。
`cg_scene.py`	场景与指令解释模块，负责解析指令、维护图元数据、执行编辑操作和导出指令，为 CLI 和 GUI 提供统一的场景状态管理。
`cg_cli.py`	命令行入口，负责读取指令文件、调用共享场景模块和保存 BMP。
`cg_gui.py`	GUI 程序入口，创建 PyQt5 应用和主窗口。
`cg_main_window.py`	GUI 主窗口，基于 Mixin 模式拆分职责：Core（选择、图层、撤销重做）、Ops（绘制、变换、裁剪）、IO（文件、模板、命令面板）。
`cg_canvas_view.py`	画布视图，处理缩放、平移、网格绘制，将交互事件委托给 `cg_interaction_modes.py` 中的状态模式类。
`cg_graphic_item.py`	图元显示对象，支持像素缓存、填充多边形渲染和图层遮挡。
`cg_dialogs.py`（入口）	对话框入口，内部实现拆分在 `_cg_dialogs/` 子包中，包含算法过程演示、算法对比、裁剪窗口等对话框。
`cg_style.py`	GUI 全局样式表，统一浅色工具界面的字体、按钮、列表、标签页和状态栏外观。

这样的划分有两个好处。第一，CLI 和 GUI 可以共用相同的指令解释逻辑，减少同一语义在两处重复实现导致的不一致。第二，核心算法模块保持独立，便于测试和报告说明，也满足课程对 `cg_algorithms.py` 依赖范围的要求。

从实现过程看，这个结构也是逐步演化出来的。项目早期只有算法函数和一个较直接的 CLI 渲染流程，GUI 原型也只是调用同一批算法函数。随着加载指令文件、导出场景、撤销重做、图层管理和命令编辑器逐步加入，如果继续让 CLI 和 GUI 各自维护图元状态，就很容易在细节上分叉，例如同一条 `rotate` 指令在两个入口中得到不同结果，或 GUI 导出的指令无法被 CLI 重新执行。因此后期将状态维护和命令分发集中到 `cg_scene.py`，让 GUI 只负责交互和显示，CLI 只负责批量执行和保存图像。

2.2 设计取舍

系统实现中有几个比较明确的取舍。

- **算法与界面分离。**核心算法函数只处理点列表和算法名，不直接依赖 Qt、Pillow 或 numpy。这样做的好处是算法可以被 CLI、GUI、测试脚本和算法演示窗口同时复用。
- **图元保存参数点，而不是保存像素点。**平移、旋转、缩放、裁剪和控制点编辑都作用在图元参数点上，绘制时再重新离散化。这样虽然每次重绘需要重新生成像素，但能避免把图元编辑退化成像素编辑。
- **GUI 撤销使用完整快照。**对于课程项目中的图元数量，完整快照的空间开销可以接受；相比命令模式，它更容易正确处理加载文件、重置画布、删除图元、多选拖拽和属性批量修改。
- **扩展功能不破坏课程指令格式。**多边形填充、显隐、锁定、操作日志和算法演示都作为 GUI 扩展存在；导出的 CLI 指令仍然围绕 README 规定的核心图元、颜色和算法展开。

2.3 数据流

系统的主要数据流可以概括为：

1. 输入层读取用户命令：CLI 从文本文件读取，GUI 从鼠标事件、菜单动作或命令编辑器读取。
2. 场景层将命令转换为图元状态：包括图元类型、参数点、颜色和算法名。
3. 算法层根据图元状态生成离散像素点。
4. 输出层将像素绘制到画布、QGraphicsScene 或图像文件中。

CLI 保存图像时，会创建白色背景的 `numpy` 数组，逐图元调用 `draw_item_pixels` 得到像素坐标，再用 Pillow 保存为 BMP。GUI 绘制时，`GraphicItem` 在 `paint` 函数中逐点调用 `QPainter.drawPoint` 显示图元，因此视觉结果与 CLI 使用同一套算法。

这种数据流还有一个实际好处：同一份输入文件可以有两种验证方式。第一种是通过 CLI 批量生成 BMP，用于自动化测试；第二种是通过 GUI 加载后观察图元列表、属性面板和画布结果，用于人工检查。自动测试关注“能否稳定生成非空图像”，GUI 则能帮助发现图层、颜色、选中状态和交互反馈方面的问题。

2.4 坐标系约定

系统统一使用屏幕坐标系：左上角为原点，x 轴向右，y 轴向下。CLI 输出图像时直接将点 (x, y) 写入数组位置 `canvas[y, x]`，不再执行上下翻转。旋转、平移和缩放也以当前屏幕坐标中的视觉效果为准。

2.5 场景状态与接口设计

系统内部没有把 GUI 操作直接绑定到某一种控件状态，而是先转换为统一的场景状态。一个图元主要由编号、类型、参数点、颜色和算法名组成；CLI 指令、GUI 鼠标操作和命令编辑器最终都会落到同一套数据结构上。这个设计并不追求一开始就做得很抽象，而是随着 CLI 和 GUI 功能逐渐增多后自然形成的：当两个入口都需要解释同一批指令、维护同一批图元时，把场景语义收拢到单独模块比继续复制代码更稳妥。其结构可简化表示为：

```
scene = {
    "width": 900,
    "height": 650,
    "pen_color": [0, 0, 0],
    "items": {
        "line1": {
            "type": "line",
            "points": [[50, 80], [320, 220]],
            "algorithm": "Bresenham",
            "color": [255, 0, 0]
        }
    }
}
```

Listing 1: 场景状态结构示意

这种设计使系统可以在多个入口之间复用能力。例如，GUI 中绘制出的图元可以导出为 CLI 指令，导出的指令又可以被 CLI 重新渲染为相同的 BMP 图像；GUI 加载指令文件时也不需要单独实现一套解析逻辑。对课程项目而言，这比只实现一个独立界面更便于测试和复现。

3 核心算法实现

3.1 线段生成算法

3.1.1 DDA 算法

DDA 算法根据线段在 x 和 y 方向上的最大变化量确定迭代步数：

$$steps = \max(|x_1 - x_0|, |y_1 - y_0|)$$

然后按如下增量迭代：

$$\Delta x = \frac{x_1 - x_0}{steps}, \quad \Delta y = \frac{y_1 - y_0}{steps}$$

每一步对当前浮点坐标四舍五入得到像素坐标。该算法直观、实现简单，适合作为线段生成的基础验证。

3.1.2 Bresenham 算法

Bresenham 算法使用整数误差项判断下一步选择哪个像素 [1]。系统实现时先判断线段是否为陡峭斜率，如果 $|dy| > |dx|$，则交换坐标轴进行统一处理。这样可以覆盖任意斜率、任意方向的线段。相比 DDA，Bresenham 的优势是避免了每步浮点累加，适合栅格设备绘图。

3.2 多边形绘制

多边形绘制没有引入独立的边界生成算法，而是将顶点按输入顺序首尾相连，每条边调用线段算法。设多边形顶点为 $P_0, P_1, \ldots, P_{n-1}$，系统依次绘制：

$$P_{n-1}P_0, P_0P_1, P_1P_2, \ldots, P_{n-2}P_{n-1}$$

这样可以自然复用 DDA 和 Bresenham。GUI 中还额外实现了扫描线多边形填充显示，用于展示扩展功能。

3.3 中点椭圆生成算法

椭圆由包围矩形的两个对角点确定。系统首先规范化输入，得到左上角和右下角，从而计算中心 (x_c, y_c) 和半径 r_x, r_y。中点椭圆算法利用四象限对称性，只需计算第一象限的点，再对称生成其他象限像素。

算法分为两个区域。第一区域中斜率绝对值小于 1，主要沿 x 方向递增；第二区域中斜率绝对值大于 1，主要沿 y 方向递减。每个区域使用判别式判断下一个像素是水平/垂直方向，还是对角方向。最后对生成点去重，避免在边界和退化情况下重复绘制。

3.4 曲线生成算法

3.4.1 Bezier 曲线

Bezier 曲线使用 de Casteljau 递推计算。对于控制点 $P_0, \ldots, P_n$，每次按参数 $t \in [0, 1]$ 线性插值得到下一层控制点：

$$P_i^{(k)}(t) = (1-t)P_i^{(k-1)}(t) + tP_{i+1}^{(k-1)}(t)$$

递推到最后一个点即为曲线上的采样点。系统根据控制多边形长度估计采样步数，并在相邻采样点之间使用 Bresenham 连线，解决采样点过稀时曲线断裂的问题。

3.4.2 三次均匀 B-spline 曲线

三次均匀 B-spline 使用连续四个控制点确定一段曲线。设局部参数为 $u \in [0,1]$，四个基函数为：

$$
\begin{aligned}
B_0(u) &= \frac{1 - 3u + 3u^2 - u^3}{6}, \\
B_1(u) &= \frac{4 - 6u^2 + 3u^3}{6}, \\
B_2(u) &= \frac{1 + 3u + 3u^2 - 3u^3}{6}, \\
B_3(u) &= \frac{u^3}{6}.
\end{aligned}
$$

曲线点为四个控制点的加权和。B-spline 曲线通常不经过首末控制点，但具有较好的平滑性，适合展示控制多边形对曲线形状的整体影响。

3.5 几何变换

系统对图元参数点执行几何变换，而不是直接变换已经离散化的像素。这样做可以避免重复离散化导致的误差累积，也能保持图元的可编辑性。

- 平移：对每个参数点执行 $(x, y) \mapsto (x + dx, y + dy)$。
- 旋转：先将点移动到旋转中心局部坐标，再使用三角函数旋转，最后平移回原坐标系并四舍五入。
- 缩放：以指定中心为基准，对参数点偏移量乘以缩放倍数。

椭圆不执行旋转，符合 README 中“不需要对椭圆进行旋转”的说明。

3.6 线段裁剪

3.6.1 Cohen-Sutherland 算法

Cohen-Sutherland 算法使用区域码判断线段端点与裁剪窗口的关系 [2]。系统将窗口外侧分为左、右、上、下四个区域，并为端点生成四位区域码。若两个端点区域码均为 0，则线段完全在窗口内；若两个区域码按位与非 0，则线段完全在窗口外；否则选择一个外侧端点，与对应窗口边界求交并更新端点。

3.6.2 Liang-Barsky 算法

Liang-Barsky 算法从线段参数方程出发 [3]：

$$
x = x_0 + u(x_1 - x_0), \quad y = y_0 + u(y_1 - y_0), \quad 0 \le u \le 1
$$

裁剪窗口四条边会给出关于 u 的约束。系统维护可见区间 $[u_1, u_2]$，若最终 $u_1 > u_2$，线段不可见；否则根据 u_1, u_2 计算裁剪后的两个端点。该方法通常能更直接地得到最终可见区间。

3.7 算法对比与适用场景

系统在 GUI 中实现了算法对比窗口，并在报告生成流程中统计算法覆盖情况。表 3 总结了主要算法的实现特点和适用场景。

表 3: 主要算法对比

对比对象	实现特点	结果差异	系统中的用途
DDA 与 Bresenham	DDA 使用浮点增量；Bresenham 使用整数误差项。	大多数线段视觉接近，但像素选择和耗时可能不同。	线段、多边形边界绘制和算法对比。
Bezier 与 B-spline	Bezier 由全体控制点共同决定；三次 B-spline 由局部四点决定每段曲线。	Bezier 经过首末控制点；B-spline 通常不经过首末控制点但更平滑。	曲线绘制、控制点编辑和曲线模板展示。
Cohen-Sutherland 与 Liang-Barsky	前者基于区域码反复求交；后者基于参数区间收缩。	两者裁剪结果应一致，计算路径不同。	线段裁剪和裁剪算法对比窗口。
Sutherland-Hodgman 多边形裁剪	逐边界裁剪多边形顶点序列，遇到跨越窗口边界的边时插入交点。	适合矩形窗口内的凸/凹多边形裁剪展示。	GUI 多边形裁剪和报告扩展说明。
扫描线填充	按扫描行求边交点并成对填充内部像素。	能直观看到多边形内部区域，而不只显示边界。	多边形填充、填充色和算法过程演示。
Bresenham 与 Wu 反走样	Bresenham 选择单个整数像素；Wu 算法为相邻像素分配亮度权重。	Wu 线段边缘更平滑，但像素和亮度处理更复杂。	线段算法对比和反走样演示。

4 命令行界面实现

命令行程序入口为 `cg_cli.py`，运行格式为：

```
python cg_cli.py input_path output_dir
```

程序首先创建场景状态，然后逐行读取输入文件。每条命令交给 `cg_scene` 中的 `apply_command` 执行。若命令为 `saveCanvas`，程序会创建白色画布，遍历当前场景中的图元，调用对应算法得到像素点并写入数组，最后保存为 BMP。

与早期版本相比，当前 CLI 的一个重要改进是场景解析逻辑独立在 `cg_scene.py` 中。这样 GUI 加载指令文件、GUI 导出指令文件以及 CLI 渲染都围绕同一套图元数据结构运行，减少了重复代码。

4.1 指令复现示例

系统保持了 README 规定的指令格式。以下片段展示了一个最小可复现流程：先重置画布并设置颜色，再绘制线段，最后保存画布。

```
resetCanvas 400 300
setColor 255 0 0
drawLine l1 50 50 350 220 Bresenham
saveCanvas demo
```

Listing 2: CLI 指令文件示例

GUI 中的“导出指令文件”功能会生成同类文本，因此 GUI 操作不是一次性的界面状态，而是可以被保存、审查和重新执行的实验记录。

5 图形交互界面实现

5.1 界面布局

GUI 使用 PyQt5 实现，主窗口采用浅色工具软件风格，由顶部命令栏、左侧工具箱、中央画布工作区、右侧侧边栏和底部状态栏组成。画布基于 `QGraphicsView` 和 `QGraphicsScene`；每个图元封装为 `GraphicItem`，内部根据算法生成像素并逐点绘制，填充多边形使用实心画刷正确遮挡下层图元。顶部命令栏使用固定尺寸图标按钮并按文件、编辑、视图、资源等类别分组；左侧工具箱使用图标按钮按选择、绘制、操作、视图分段，提供选择、线段（含算法弹出菜单）、多边形（含算法弹出菜单）、椭圆、Bezier、B-spline、裁剪、交互旋转、交互缩放、适配画布和 100% 缩放。中央画布放在浅灰工作区内，白色画布纸张带细边框和淡色自适应网格，支持滚轮缩放和 Space/中键拖拽平移。右侧侧边栏划分为图层、属性和命令三个主标签页，其中命令页内部划分为指令编辑和操作日志。底部状态栏显示当前模式、鼠标坐标和缩放比例。每次关键操作完成后，画布底部会短暂显示中文操作字幕；字幕内容不只是功能名，还会说明算法含义、操作结果或边界情况，并支持自动换行和较长停留时间，便于录制演示视频时直接作为画面反馈。

界面布局的目标不是做一个复杂的专业绘图软件，而是让课程要求中的算法、图元和编辑操作能够被清楚地看到和操作。左侧工具箱承担“当前正在做什么”的模式切换，右侧面板承担“当前选中了什么、它有哪些参数”的信息展示，顶部工具栏保留文件、撤销重做、保存和模板等高频入口。这样在演示时可以沿着“绘制 - 选择 - 编辑 - 对比 - 导出”的顺序讲解，不需要频繁在不同窗口或命令行之间切换。

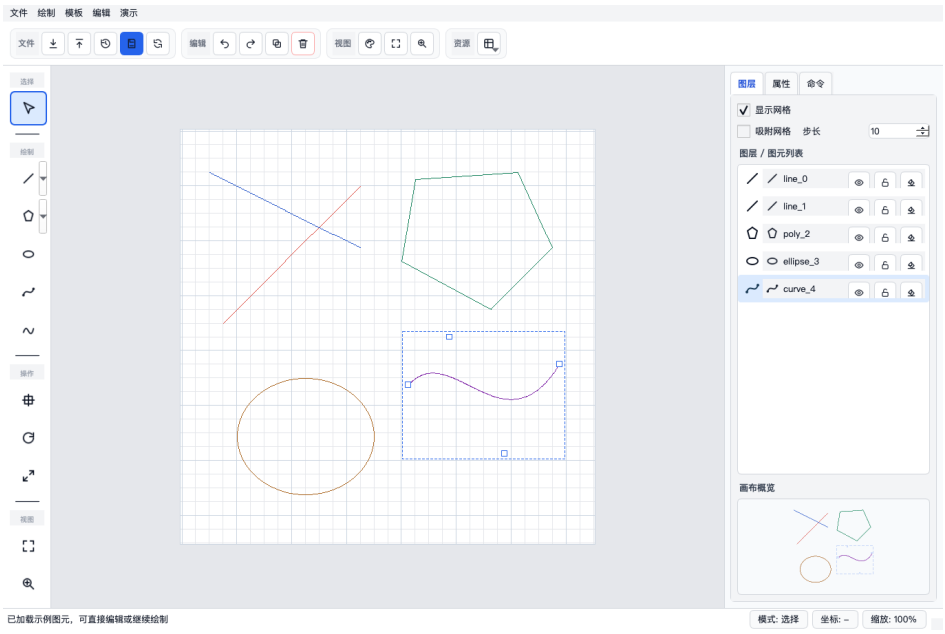


图 1: GUI 界面示意图

5.2 绘制交互

系统将不同图元的绘制方式设计为符合直觉的鼠标操作：

- 线段与椭圆：左键拖拽确定两个端点，移动过程中显示预览。
- 多边形与曲线：左键逐点输入顶点或控制点，右键结束当前图元。
- B-spline 曲线：要求至少四个控制点。
- 坐标输入会被限制在当前画布范围内；开启吸附网格时，输入点会被吸附到指定步长。

绘制过程中的预览图元只存在于当前交互模式中，不会进入正式图元列表。线段和椭圆在拖拽过程中实时显示当前端点；多边形和曲线在右键结束前以折线预览已有控制点和当前鼠标位置。这样用户能够在确认之前看到图元的大致形状，避免每一次输入错误都必须删除图元后重新开始。

5.3 对象选择、属性面板与编辑

系统支持图层列表选择、画布点击选择、框选和 Shift 追加选择。鼠标悬停到可选图元时，会显示浅蓝色虚线框；图元选中后，会显示外接矩形和控制点。用户可以直接拖动图元进行平移，也可以拖动控制点修改参数。右侧属性面板以表格形式展示编号、类型、算法、颜色、可见性、锁定状态、填充状态和参数点，并使用坐标表格直接编辑控制点。图层列表每行提供类型图标以及显隐、锁定和填充快捷按钮。菜单和右键上下文菜单还提供复制、重命名、改色、显示隐藏、锁定、填充、交互式旋转、交互式缩放、线段裁剪、多边形裁剪、交互式裁剪、删除和图层调整等操作。

为了提高可恢复性，系统实现了撤销和重做。每次改变场景的操作前保存完整场景快照。相比只保存单个图元状态，完整快照更适合处理加载场景、删除图元、重置画布、多选编辑等复杂操作。

属性面板的作用不只是显示信息，也承担了部分“精确编辑”的职责。对于线段、椭圆、多边形和曲线，用户可以在参数点表格中查看并修改每个点的坐标；对于线段、多边形和曲线，也可以在算法下拉框中切换当前算法。这样既保留了鼠标交互的直观性，又保留了坐标级修改的精确性。

5.4 交互式变换与视图反馈

除表单化参数输入外，GUI 还提供交互式旋转和交互式缩放。进入交互式变换后，画布上会显示可拖动的变换中心点；用户可以先拖动中心点调整旋转或缩放中心，再在画布上拖拽生成实时预览。旋转模式会显示当前角度，缩放模式会显示当前比例，松开鼠标后应用结果，右键可取消当前操作。该设计比纯文本输入更适合演示几何变换的视觉效果。

视图层面，系统支持鼠标滚轮缩放、空格加左键拖拽或鼠标中键平移、适配画布和 100% 缩放。右侧图层页提供“画布概览”小地图，用于在较大画布中快速确认整体布局。网格显示、吸附网格和吸附步长变化也会给出字幕反馈，说明这些开关只影响辅助定位而不改变图元本身。保存画布时，系统会临时隐藏选中框、悬停框、预览图元、交互裁剪辅助框和交互变换辅助线，只保留实际图形内容。

5.5 模板、演示与报告导出

最终版本中加入了多组模板场景，包括综合示例、裁剪示例、曲线示例和变换示例。模板通过工具栏下拉菜单一键加载，加载后会替换当前场景并刷新指令编辑器。综合示例适合快速展示基础图元，裁剪、曲线和变换模板则适合在演示视频中分别说明专项功能。

算法演示窗口可以逐步播放 DDA、Bresenham、中点椭圆、裁剪、Bezier de Casteljau、扫描线填充和 Wu 反走样线段过程。算法对比窗口能够并排展示两种算法结果，并统计像素数量和运行耗时；其中线段默认对比 Bresenham 与 Wu 反走样，曲线对比 Bezier 与 B-spline，多边形对比 DDA 与 Bresenham 边界生成。裁剪算法对比窗口对同一条线段使用同一裁剪窗口，对比 Cohen-Sutherland 与 Liang-Barsky 的结果。

此外，系统支持导出操作历史，内容包括导出时间、画布尺寸、当前可复现 CLI 指令和 GUI 操作日志。命令编辑器带行号显示，用户可以直接修改 CLI 指令并点击保存按钮；系统会先解析全部文本，成功后替换当前场景，失败时高亮错误行并报告原因，同时保留原画布不被错误指令覆盖。该功能用于保留演示过程和调试过程，也能帮助检查 GUI 场景是否仍可被文本指令复现。

5.6 命令编辑器与操作历史

GUI 中的命令编辑器是连接交互操作和文本指令的关键部分。普通绘图软件往往只保存内部格式，而本系统的一个目标是让 GUI 生成的结果仍然能够回到课程要求的指令文件中。因此每当场景变化时，系统会根据当前可见图元重新生成一段 CLI 指令，包括 `resetCanvas`、必要的 `setColor` 和各类 `draw*` 命令。用户可以直接查看这些指令，也可以在编辑器中手动修改后重新应用。

操作历史与导出指令文件的侧重点不同。导出指令文件只描述最终场景，而操作历史会记录“怎样到达这个场景”，例如加载了哪个模板、绘制了哪个图元、是否执行过交互式裁剪、是否改过颜色或调整过图层。这个功能对调试和最终演示都有帮助：如果一次演示效果较好，可以把操作过程保存下来；如果出现问题，也能通过日志回看问题大致发生在哪一步。

6 测试与结果

6.1 测试方法

系统提供回归测试脚本 `run_regression_tests.py`。脚本会自动扫描 `source` 目录下的 `input_*.txt`，对包含 `saveCanvas` 的输入文件运行 CLI，检查生成图片数量、图片尺寸和非白像素数量。带 `--report-dir` 参数时，脚本还会生成 HTML 和 Markdown 测试报告。

测试命令如下：

```
python source/run_regression_tests.py --output-dir /tmp/cg_outputs_final
python source/run_regression_tests.py --report-dir /tmp/cg_report_final
```

当前测试结果为：4 个输入文件中，3 个包含 `saveCanvas`，共生成 13 张非空 BMP 图像；`input_april_gui_demo.txt` 因没有保存指令而被跳过。测试覆盖 DDA、Bresenham、Bezier、B-spline、中点椭圆、Cohen-Sutherland 和 Liang-Barsky。

6.2 测试结果图

图 2 至图 5 展示了 5 月综合测试的输出结果。

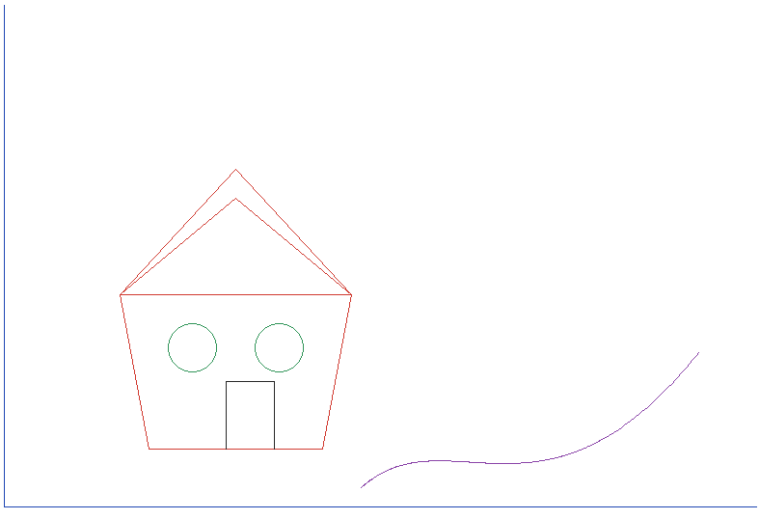


图 2: 综合场景：线段、多边形、椭圆和 Bezier 曲线

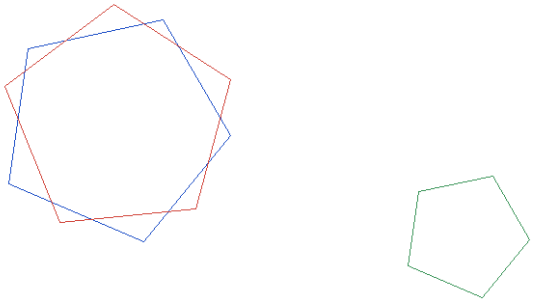


图 3: 几何变换对比：原始多边形、旋转和缩放平移

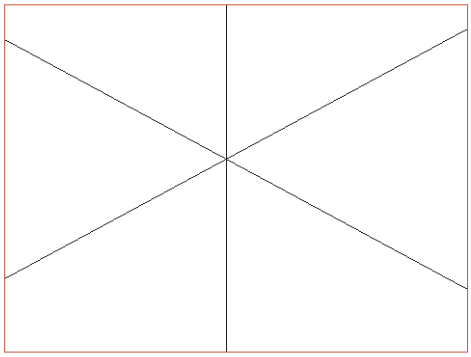


图 4: 线段裁剪对比：Cohen-Sutherland 与 Liang-Barsky

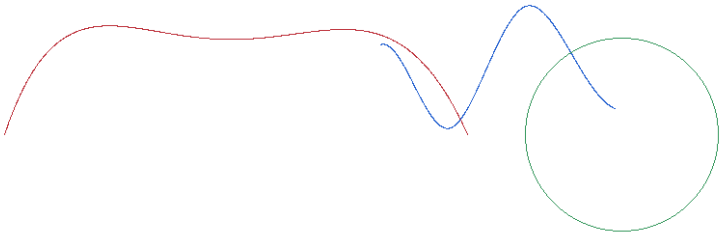


图 5: 曲线与椭圆：高阶 Bezier、B-spline 和反向包围框椭圆

6.3 测试覆盖分析

表 4 列出主要测试文件和验证内容。

表 4: 测试输入与覆盖内容

输入文件	覆盖内容
`input_march.txt`	基础线段、多边形、平移、缩放、椭圆、裁剪、Bezier 和 B-spline。
`input_april.txt`	水平线、竖直线、负斜率线、退化线段、反向椭圆包围框、裁剪边界情况和曲线。
`input_may.txt`	综合场景、变换对比、裁剪对比、高阶曲线和反向包围框椭圆。
`input_april_gui_demo.txt`	GUI 加载指令文件演示，无 `saveCanvas`，因此回归测试中不生成图片。

从测试结果看，系统能够稳定处理课程要求中的典型输入和一些边界情况。特别是退化线段、反向输入的椭圆包围框、完全在窗口外的裁剪线段等情况均能正常处理。

表 5: 回归测试输出统计

输入文件	输出图数	非白像素数量	覆盖算法
`input_march.txt`	5	962, 1260, 1779, 1533, 557	DDA、Bresenham、Bezier、B-spline、Ellipse、Cohen-Sutherland、Liang-Barsky。
`input_april.txt`	4	1733, 2004, 822, 812	退化线段、边界裁剪、反向椭圆包围框和曲线绘制。
`input_may.txt`	4	3372, 1645, 2994, 1716	综合场景、变换前后对比、双算法裁剪对比和曲线椭圆组合。
合计	13	全部非空	覆盖 README 要求的全部核心图元、变换和裁剪算法。

7 遇到的问题与解决方案

7.1 坐标系一致性

早期实现参考了旧版本示例中的坐标习惯，容易在输出图像时出现上下翻转或旋转方向不一致的问题。最终系统统一为左上角原点、y 轴向下，并在 CLI 输出时直接写入 `canvas[y,x]`。这样 CLI 输出、GUI 显示和题目说明保持一致。

7.2 曲线采样断裂

Bezier 和 B-spline 都需要离散采样。如果只绘制采样点，当曲线较长或控制点间距较大时会出现断裂。为解决这个问题，系统在相邻采样点之间调用 Bresenham 连线，使曲线视觉上连续。

7.3 CLI 与 GUI 逻辑重复

3 月早期版本中，CLI 和 GUI 各自维护一部分场景逻辑。随着功能增加，这种做法容易产生不一致。4 月开始抽取 `cg_scene.py`，将指令解析、图元构造和编辑操作统一封装，使 CLI 与 GUI 共用同一套场景语义。

7.4 撤销粒度

GUI 中撤销操作如果只记录单个图元，很难处理重置画布、加载文件、删除图元、多选编辑等操作。因此最终采用完整场景快照的方式。虽然空间开销略大，但实现清晰，并且当前图元规模下性能足够。

8 扩展功能与系统亮点

除了课程要求的基本功能外，系统还实现了以下扩展：

- GUI 采用现代浅色工具界面，提供分组图标命令栏、左侧分段工具箱、中央画布工作区、画布自适应网格、右侧图层/属性/命令面板和底部状态栏。
- GUI 支持网格显示、吸附网格、坐标提示、缩放比例显示、画布概览、图元属性面板、坐标表格编辑、图层行内快捷按钮、图层调整、锁定和显隐。
- 支持直接拖拽图元和控制点，提升交互编辑效率。
- 支持右键上下文菜单，能够快速执行复制、重命名、改色、隐藏、锁定、填充、图层调整、裁剪和删除等操作。
- 支持交互式旋转和交互式缩放，可拖动变换中心并实时预览角度或比例。
- 支持必要的参数化输入，重置画布和裁剪窗口不依赖纯文本输入；平移、旋转和缩放则主要通过鼠标拖拽和交互式预览完成。
- 支持带行号命令编辑器，可以在 GUI 中修改 CLI 指令并一键解析应用，出错时高亮错误行。
- 支持模板加载与交互展示，可快速切换综合、裁剪、曲线和变换场景，便于录屏说明不同功能。
- 支持详细操作字幕反馈，覆盖绘制、选择、编辑、变换、裁剪、图层、文件、网格吸附和演示窗口等操作，便于录屏展示。
- 支持算法过程演示和算法对比，包含 Bezier de Casteljau 动画、扫描线填充、Sutherland-Hodgman 多边形裁剪和 Wu 反走样线段。

- 支持导出操作历史，记录 GUI 操作和可复现指令。
- 支持批量测试可视化报告，展示算法覆盖和输出图像。

这些功能虽然不全是课程基础指令格式的一部分，但能让系统更像一个完整的二维绘图工具，也方便最终报告和演示视频展示。

9 最终交付与复现方式

最终提交内容围绕“程序、说明、报告、演示”组织。程序源码保留在 `source` 目录中；用户说明书用于解释安装、运行和 GUI 操作；本系统报告用于说明设计、算法和测试；演示视频用于展示最终系统效果。

表 6: 最终交付文件说明

文件或目录	说明
`source/`	Python 源码，包括核心算法、CLI、GUI、场景管理和测试脚本。
`231220066_说明书.pdf`	用户说明书，说明环境配置、命令行运行、GUI 操作和扩展功能。
`231220066_报告.pdf`	系统报告，即本文档，说明系统结构、算法实现、测试结果和个人思考。
`231220066_月报告.pdf`	阶段性月报告，记录 3 月、4 月、5 月和最终阶段的开发过程。
`231220066_演示.mp4`	演示视频，展示 CLI、GUI、综合/裁剪/曲线/变换模板、算法对比和测试报告。

在本地复现实验时，可以在项目根目录执行以下命令：

```
python3 source/run_regression_tests.py --output-dir /tmp/cg_outputs
python3 source/run_regression_tests.py --report-dir /tmp/cg_report
python3 source/cg_gui.py
```

Listing 3: 测试与报告复现命令

第一条命令用于验证所有带 `saveCanvas` 的输入文件能生成非空 BMP；第二条命令生成可视化回归测试报告；第三条命令启动 GUI 以检查模板场景和交互功能。

10 个人思考

本次项目最大的体会是：图形学算法本身往往可以在较小的函数中实现，但要做成一个完整系统，还需要处理大量算法之外的问题，例如坐标系约定、交互状态管理、图元选择、撤销恢复、文件复现和测试验证。

在线段、椭圆和曲线算法中，离散化细节会直接影响最终视觉效果。比如曲线采样点之间是否连线，决定了曲线是否连续；旋转后是否四舍五入，也会影响图元位置是否稳定。在线段裁剪中，Cohen-Sutherland 的区域码思路直观，Liang-Barsky 的参数区间更紧凑，两者都适合报告和演示中进行对比。

系统结构上，我认为将 `cg_algorithms.py`、`cg_scene.py`、CLI 和 GUI 分开是比较关键的设计。核心算法保持纯净，场景层统一指令语义，界面层只负责交互和展示，这样后续扩展功能时不容易破坏原有测试。

如果继续改进，可以考虑加入更完整的图元持久化格式、更多填充算法、曲线局部编辑约束、导出矢量格式，以及更细致的性能统计。不过就课程要求而言，当前系统已经覆盖主要算法和完整交互流程。

11 结论

本项目完成了课程要求的二维绘图系统。核心算法模块实现了线段、多边形、椭圆、曲线、几何变换、线段裁剪、多边形裁剪、扫描线填充和反走样线段等能力；命令行程序可以按指令文件生成 BMP 图像；GUI 程序支持完整的鼠标交互绘制和编辑，并在此基础上加入了交互式多边形裁剪、详细操作字幕反馈、图层遮挡管理、模板场景系统、算法过程演示、算法并排对比、命令编辑器和可视化回归测试报告等扩展功能。

通过多组输入文件和批量回归测试，系统能够稳定生成非空图像，覆盖课程要求的主要算法。最终系统既能满足自动测试，也能作为图形交互程序进行展示和使用。

参考文献

[1] J. E. Bresenham. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 4(1):25–30, March 1965.

[2] Robert F. Sproull and William M. Newman. *Principles of Interactive Computer Graphics*. McGraw–Hill, New York, 1973.

[3] You-Dong Liang and Brian A. Barsky. A new concept and method for line clipping. *ACM Transactions on Graphics*, 3(1):1–22, January 1984.